Data Structures - Lecture Notes

1. Arrays and Linked Lists

An array stores elements in contiguous memory locations. Because the address of any element can be computed from its index, reading or writing an element by index takes constant time, O(1). The drawback is that the size is usually fixed at creation, and inserting or deleting an element in the middle requires shifting all later elements, which takes O(n) time.

A linked list stores each element in a node that contains the data and a pointer to the next node. In a singly linked list each node points only forward; in a doubly linked list each node also keeps a pointer to the previous node, which makes deletion and backward traversal easier at the cost of extra memory. Nodes can be anywhere in memory, so the list grows and shrinks easily.

Inserting or deleting a node is O(1) once the position is known, because only a few pointers change and no elements are shifted. However, finding the position requires walking from the head, so accessing the i-th element is O(n). Linked lists also use extra memory for pointers and have poor cache locality compared with arrays. In practice, choose arrays when random access is frequent and linked lists when insertions and deletions at known positions dominate.

2. Stacks and Queues

A stack is a linear structure that follows the Last-In First-Out (LIFO) rule. The main operations are push, which adds an element on top, pop, which removes the top element, and peek, which reads it. Stacks are used for the function call stack, undo features, checking balanced parentheses and evaluating postfix expressions.

A queue follows the First-In First-Out (FIFO) rule. Elements are added at the rear with enqueue and removed from the front with dequeue. Queues appear in task scheduling, print spoolers and buffering of data streams.

In a simple array-based queue, the front index keeps moving forward, so space freed at the start of the array cannot be reused even though the queue is not full. A circular queue fixes this by treating the array as a ring: when the rear index reaches the end it wraps around to index zero using modulo arithmetic, so freed slots are reused. The queue is full when the next position of the rear equals the front, and empty when front equals rear.

A deque (double-ended queue) allows insertion and removal at both ends, and a priority queue removes the element with the highest priority first, usually implemented with a heap.

3. Trees

A tree is a hierarchical structure made of nodes connected by edges, with one root and no cycles. In a binary tree every node has at most two children, called left and right. A binary search tree (BST) adds an ordering rule: for every node, all keys in the left subtree are smaller and all keys in the right subtree are larger than the node's key. This allows search, insertion and deletion in time proportional to the height of the tree.

There are three depth-first traversals. Inorder visits left subtree, node, then right subtree, and on a BST it produces the keys in sorted ascending order. Preorder visits the node first, then left and right, and is useful for copying a tree. Postorder visits the children before the node and is used when deleting a tree. Level-order traversal visits nodes level by level using a queue.

A plain BST can become skewed, for example when keys are inserted in sorted order, and then its height grows to n so operations degrade to O(n). An AVL tree is a self-balancing BST that avoids this. The balance factor of a node is the height of its left subtree minus the height of its right subtree, and in an AVL tree it must be -1, 0 or +1 for every node. When an insertion or deletion breaks this rule, the tree restores balance using rotations: single left, single right, left-right and right-left. As a result, the height stays O(log n) and all main operations take O(log n) time.

4. Hashing and Graphs

A hash table stores key-value pairs and uses a hash function to convert a key into an array index, giving average O(1) lookup, insertion and deletion. Two different keys can map to the same index, which is called a collision. The load factor is the number of stored entries divided by the number of buckets; as it grows, collisions become more frequent, so tables are resized when the load factor passes a threshold.

In separate chaining, each bucket holds a linked list (or another small collection) of all entries that hash to that index, so colliding keys are simply appended to the chain. In open addressing, all entries live inside the array itself and a collision is resolved by probing for another empty slot, using linear probing, quadratic probing or double hashing.

A graph consists of vertices connected by edges and can be stored as an adjacency matrix or adjacency lists. Breadth-first search (BFS) explores vertices level by level starting from a source and uses a queue; it finds shortest paths in unweighted graphs. Depth-first search (DFS) goes as deep as possible along one branch before backtracking and uses a stack, either explicitly or through recursion. Dijkstra's algorithm finds the shortest paths from one source in a weighted graph when all edge weights are non-negative, by repeatedly picking the unvisited vertex with the smallest known distance, typically with a priority queue.